

LLM Inference Architecture

LLM이란

대규모 언어 모델(Large Language Model)은 어떤 텍스트가 주어졌을 때 **다음에 올 단어의 확률 분포를 예측**하는 수학적 함수.

하나의 단어를 확정적으로 찍어내는 것이 아니라, **가능한 모든 다음 단어에 대해 확률을 계산함**.

이때 가장 확률이 높은 단어만 고르지 않고, 가끔 확률이 낮은 단어도 랜덤하게 선택(샘플링)하면 더 자연스러운 답변이 됨.

모델 자체는 결정론적(deterministic)이지만, 이 샘플링 덕분에 같은 입력에도 매번 다른 답변이 나올 수 있음.

Training (학습) workload

분산 AI 학습은 데이터센터 네트워크에 가장 큰 부하를 주는 작업. 수백~수천 개 GPU가 매 iteration마다 gradient와 파라미터를 동기화하며 lockstep으로 동작해야 함.

구체적으로, 512개 GPU에 분산된 LLM 학습을 예로 들면 — 각 GPU가 입력 데이터의 일부를 처리한 뒤, 매 배치마다 모든 GPU가 gradient를 교환하는 all-reduce 연산을 수행. 이때 가장 느린 연결이 전체 작업의 속도를 결정함. 대역폭 부족, 오버서브스크립션, 스위치 설정 오류 하나만으로 클러스터 전체 활용률이 급락할 수 있으며, 이는 곧 수백만 달러의 GPU 투자 낭비로 직결됨.

네트워크 요구사항:

- 고처리량 all-to-all 통신을 최소 지터로 지원
- 논블로킹·무손실 패브릭 필수
- 작은 비효율이라도 전체 학습 시간과 비용에 직접적 영향

Inference (추론) workload

기본 추론

소형·양자화 모델의 경우 stateless하게 수평 확장이 가능 — 이미지 분류, 간단한 챗봇 등은 범용 서버에서 로드밸런싱하면 충분. 그러나 70B 파라미터 이상의 대형 모델은 하나의 추론 요청을 처리하는 데에도 여러 GPU가 협력해야 하며, GPU 간 고속 백엔드 링크가 필요함.

학습의 all-to-all 동기화와 달리, 대형 모델 추론은 파이프라인 형태의 통신이 주를 이룸 — 순차적이고, 지연에 민감하며, 모델 아키텍처에 따라 통신 패턴이 크게 달라짐.

Agentic/Reasoning 모델의 등장

최근 agentic 및 reasoning 모델의 등장으로 추론의 성격이 근본적으로 변화함. 더 이상 단일 프롬프트 → 응답 구조가 아니라, 하나의 사용자 요청이 다음을 유발하는 구조:

- 내부 모델 호출의 연쇄
- 외부 API 쿼리
- 다단계 추론 체인

이로 인해 추론이 stateful·세션 기반이 되며, 복잡한 오케스트레이션과 지속적 컨텍스트 유지가 필요해짐.

Stateless와의 핵심 차이 — stateless 환경에서는 서버를 늘리고 로드밸런서에 맡기면 해결됨. 반면 agentic 추론에서는 수십 개의 내부 스텝과 여러 모델에 걸쳐 컨텍스트를 유지해야 함. 세션 어피니티가 없으면 캐시된 컨텍스트를 잃고 지연이 급증.

Transformer Model

2017년 이전까지 대부분의 모델은 단어를 순차적으로 처리했으나, 구글이 발표한 트랜스포머는 전체 문장을 한꺼번에 병렬로 처리할 수 있게 한 구조.

핵심 구성 요소는 두 가지.

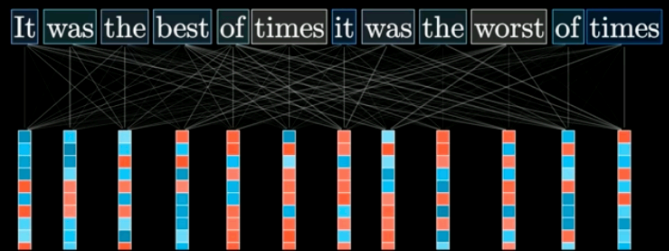
- **어텐션(Attention)** — 각 단어의 숫자 벡터가 서로 정보를 주고받으며, 주변 맥락에 따라 의미를 조정하는 연산 메커니즘. "눈"이라는 단어가 "내린다" 옆에 있으면 날씨의 눈, "보는" 옆에 있으면 신체 기관의 눈으로 해석되는 식.
- **FeedForward network** — 모델이 더 많은 언어 패턴을 저장할 수 있도록 하는 연산 메커니즘.

Attention과 FeedForward를 여러 레이어에 걸쳐 반복하면 각 단어 벡터는 점점 더 풍부한 맥락을 반영하게 되고, 마지막 단계에서 전체 문맥을 반영한 벡터로 다음 단어의 확률 분포를 예측.

Previous Models 이전 모델



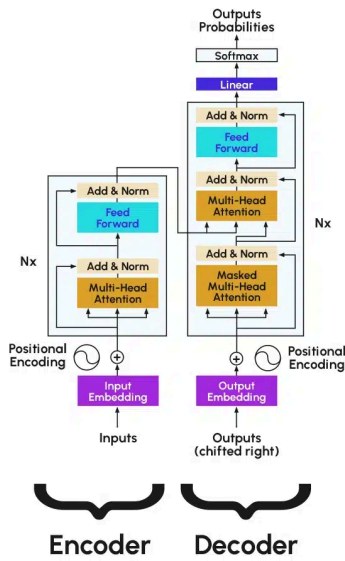
트랜스포머 Transformers



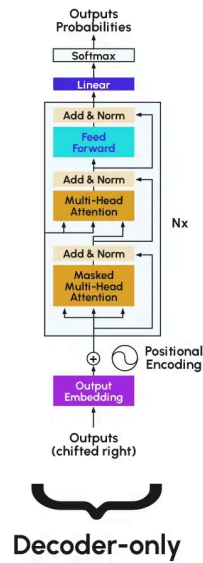
단어를 하나씩 순차적으로 처리 ⇒ 전체 문장을 한꺼번에 병렬로 처리

Transformer model 별 사용 용도

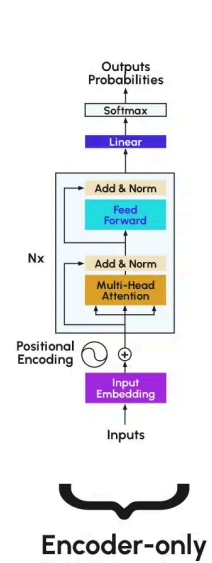
Transformer



GPT*



BERT*



*Illustrative example, exact model architecture may vary slightly

	Transformer	GPT	BERT
한줄 요약	문장을 병렬로 처리하는 기본 아키텍처	Transformer 디코더 로 다음 단어를 예측하는 모델	Transformer 인코더 로 문장 전체를 이해하는 모델
구조	인코더(입력 이해) + 디코더(출력 생성) 결합	디코더만 사용	인코더만 사용
읽는 방향	인코더는 양방향, 디코더는 단방향	왼쪽 → 오른쪽 단방향	양방향 (앞뒤 문맥 동시에 참조)
동작 방식	입력 문장을 인코더가 이해한 뒤, 디코더가 출력을 한 단어씩 생성	앞에 나온 단어들만 보고 그다음 단어를 반복적으로 예측하여 텍스트를 이어 붙임	문장 내 일부 단어를 가린 뒤, 앞뒤 문맥을 동시에 보고 가려진 단어를 맞추는 방식으로 학습
이 방식의 장점	입력과 출력 길이가 다른 변환 작업(번역 등)에 적합	단어를 순차적으로 이어 붙이므로 자연스러운 긴 텍스트 생성이 가능	양방향으로 문맥을 파악하므로 문장의 의미를 더 정확하게 이해
잘하는 일	번역, 요약 등 입력 → 출력 변환	글쓰기, 대화, 코드 생성	감정 분석, 질의응답, 문장 분류
대표 사례	Claude	구 ChatGPT, GitHub Copilot	Google 검색 랭킹, 스팸 필터

KV Cache 정의와 역할

LLM이 문장을 생성할 때 self-attention의 계산 복잡도는 시퀀스 전체 길이 L 에 대해 $O(L^2)$.

⇒ 이대로라면 단어 하나를 생성할 때마다 매번 $O(L^2)$ 을 계산해야 하는데, 이것이 KV Cache가 해결하는 문제.

KV Cache는 어텐션* 연산에서 사용되는 Key와 Value 벡터를 저장해두는 캐시.

한 번 $O(L^2)$ 계산으로 Key, Value를 만들어 캐시에 저장해두면, 이후 단어 생성부터는 새로 추가된 단어 하나에 대해서만 기존 캐시와 어텐션을 계산하면 되므로 단어 하나당 $O(L)$ 로 복잡도가 줄어듦.

정리하면,

- KV Cache가 없으면 N 개의 단어를 생성하는 데 $O(N \times L^2)$ 이 필요하지만
- KV Cache를 사용하면 초기 1회 $O(L^2)$ 이후에는 **각 단어당 $O(L)$ 만 필요**.

Prefill과 Decode의 차이

LLM 서빙에서 추론은 크게 두 단계로 나뉨.

- 추론
 - **Prefill (채우기 단계)** — 입력 시퀀스 전체를 한꺼번에 처리하여 **KV Cache**를 구축하는 단계. $O(L^2)$ 계산이 발생하며, 이 단계에서는 아직 새로운 단어를 생성하지 않음. 입력 토큰들을 병렬로 처리할 수 있으므로 GPU 활용도가 높고 compute-bound 작업.
 - **Decode (생성 단계)** — KV Cache가 준비된 상태에서 단어를 하나씩 순차적으로 생성하는 단계. 매 스텝마다 새 토큰 하나에 대해서만 연산하고 결과를 KV Cache에 추가함. 단어 하나당 $O(L)$ 이며, 각 단어가 이전 단어에 의존하므로 병렬화가 불가능하고 memory-bound 작업.

Prefill 최적화: 시스템 프롬프트와 Prefill Chunk

서빙 환경에서 prefill의 부담을 줄이는 두 가지 기법.

- **시스템 프롬프트 사전 계산** — 시스템 프롬프트는 사용자 입력이 들어오기 전부터 이미 알고 있는 고정된 시퀀스. 따라서 서빙 전에 미리 KV Cache를 계산해둘 수 있음. 전체 길이 L 을 시스템 프롬프트 길이 L_p 와 사용자 입력 길이 L_u 로 나누면, $O(L_p^2)$ 은 사전에 처리하고 서빙 시에는 $O(L_u^2)$ 만 계산하면 됨. 시스템 프롬프트가 아무리 길어도 서빙 지연에 영향을 주지 않는 구조.

- **Prefill Chunk** — 사용자 입력(L_u)은 미리 알 수 없으므로 사전 계산이 불가능. 그렇다고 매우 긴 입력에 맞춰 GPU를 준비하면 짧은 입력이 대부분인 상황에서 낭비가 심하고, 짧은 입력에만 맞추면 긴 입력을 처리할 수 없음.

이를 해결하는 것이 prefill chunk 방식.

사용자 입력을 고정된 chunk 크기(일반적으로 128 또는 512 토큰)로 나눠서 순차적으로 KV Cache를 구축함.

예를 들어 chunk size가 512라면, GPU는 한 번의 inference에서 512^2 복잡도만 감당하면 되고, 이를 반복하여 전체 입력의 KV Cache를 완성.

GPU 메모리와 계산량을 예측 가능한 범위 내로 통제할 수 있는 것이 핵심.



전체 서빙 흐름 요약

1. 시스템 프롬프트에 대한 KV Cache를 서빙 전에 미리 계산(prefill)
2. 사용자 입력이 들어오면 chunk 단위로 나눠서 KV Cache를 순차 구축(prefill chunk)
3. 전체 입력에 대한 KV Cache가 완성되면 decode 단계 진입
4. 한 토큰씩 생성하며 KV Cache에 추가, 응답이 완료될 때까지 반복